

Speeding Up Exact Unknot Recognition

One-sided modular filters, sparse cancellation, and compact connected-sum factors

Implementation and technical analysis accompanying fastunknot 0.2.0

18 September 2026

Abstract

We improve the Python implementation supplied in the `fast/` directory of `Knots.zip`, preserving an exact and complete Khovanov-based fallback. The main changes are exact finite-field Alexander and Jones rejection filters, immutable memoized cobordism arithmetic, lazy fill-aware cancellation, faster scan ordering and descending tests, and certified diagrammatic connected-sum factorization. On the supplied 36-crossing random five-braid, the original engine exceeds a 60-second same-machine budget; the optimized unfactored rank computation completes in 1.110 seconds. With caching but the old LIFO pivot strategy, the same task takes 49.576 seconds. Default recognition of the Conway and Kinoshita–Terasaka examples is respectively 21.1 and 24.5 times faster.

The general worst-case bound remains $2^{O(n)}$. We do not claim to implement Lackenby’s general quasi-polynomial algorithm. A proved improvement is $\text{poly}(n) + \sum_i 2^{O(n_i)}$, where n_i are the sizes of the diagrammatic connected-sum factors actually found. Consequently the implementation is polynomial when their maximum size is $O(\log n)$ and quasi-polynomial when it is $O(\log^2 n)$. A sum of 100 trefoils is processed in 65.6 milliseconds, with at most 18 live objects, while its reduced rank 3^{100} is represented exactly. The deliverable includes the original baseline, runnable source, certificates, 30 regression-test methods, and an exhaustive comparison on 2856 small closed braid words against structurally independent computations, with no mismatches.

Contents

1 Outcome, scope, and the supplied baseline	3
1.1 What is and is not changed	3
2 What the Lackenby material does and does not supply	3
3 Execution architecture and exactness contract	4
4 Exact one-sided modular rejection filters	5
4.1 Evaluating the Alexander minor without polynomial elimination	5
4.2 Jones evaluations as a matching-valued transfer computation	5
4.3 Writhe and independently checkable rejection records	6
5 Faster cobordism arithmetic and cancellation	7
5.1 The retained local algebra	7
5.2 Every unit is its own inverse	7
5.3 Bounded memoization and immutable coefficients	7
5.4 Sparse, lazy Markowitz-style cancellation	8
6 Certified diagrammatic connected-sum factorization	8
6.1 Recognizing a valid cut without a prime-decomposition oracle	8
6.2 Why the rank product and degree convolution are exact	9
6.3 Bounds and a genuine asymptotic improvement	10
7 Faster polynomial preprocessing	11
7.1 An equivalent greedy order in $O(n \log n)$ per start	11
7.2 All descending basepoints by one interval sweep	11
8 Validation and its limits	11
9 Same-machine experiments	12
9.1 Methodology	12
9.2 Raw homology calculations	13
9.3 The large five-braid: separating caching from cancellation	13
9.4 Default recognition, including regressions	14
9.5 Compact products instead of exponentially many objects	14
9.6 Ordering alone	14
10 Using and reproducing the deliverable	15
11 Remaining limits and concrete next implementation targets	16

1 Outcome, scope, and the supplied baseline

The objective was first to pursue the quasi-polynomial target $n^{O(\log n)}$, and otherwise to improve the supplied recognizer in a verifiable way. The delivered program achieves the latter, together with a stronger bound on an explicitly characterized class of diagrams. There is no unimplemented oracle on its execution path. It uses only Python’s standard library and runs without network access or an external topology package.

Throughout, n is the number of crossings in a validated, one-component, classical planar diagram. Edge labels are compacted; after compaction the input has $O(n \log n)$ bits. Reading arbitrarily long user-supplied integer labels is an additional input-encoding cost, not a function of n alone. Complexity bounds assume sufficient memory and no user-imposed timeout. The resource-limited interface deliberately permits the third answer UNKNOWN.

The original program is preserved under `baseline/`; the improved version is under `fast/`. The original program already had several important components: planar-diagram, braid, and grid input; checked decreasing R1/R2 moves; a descending-diagram test; an exact Alexander polynomial; and a scanning Khovanov calculation over $\mathbb{F}_2$ using local dotted cobordisms. This is a useful starting point, not a naive full-cube enumerator. The improvement therefore concentrates on avoiding unnecessary homology computations, reducing repeated algebra, reducing cancellation fill-in, and avoiding explicit tensor-product multiplicities.

The supplied historical benchmark was produced on a different Windows/Python installation. It records a 600-second timeout on the large five-braid. That number is useful for identifying a difficult test, but is *not* used in any same-machine speedup ratio in this report. The new measurements and their individual samples are in `results/benchmark.json`. Input and source hashes are provided so that the comparison can be repeated.

1.1 What is and is not changed

The topological meaning of accepted inputs is unchanged. The recognizer still uses exact arithmetic. The final decision criterion is still total reduced $\mathbb{F}_2$ Khovanov rank one. It does not compute integral torsion or the full quantum bigrading. The returned degree histogram is in *unshifted cube degree*; this distinction matters when comparing diagrams related by Reidemeister moves.

The default execution method may now report a modular Alexander or Jones rejection instead of a full polynomial or homology computation. These are exact one-sided implications, not randomized guesses. The low-level rank API now also validates the PD and any explicitly supplied crossing order. The writhe implementation is corrected to use both oriented branches, while the original local signs used in the Fox matrix retain their original convention.

2 What the Lackenby material does and does not supply

The supplied talk announces a general quasi-polynomial unknot-recognition algorithm; its theorem slide attributes the result to 2021. Its later slides identify four acceleration ingredients: surfaces of quadratic pattern complexity, compressed hierarchies, multisurfaces, and Heegaard splittings with Cheeger regions. The final flowchart contains operations on these objects, not on Khovanov complexes [1].

The supplied July 2026 paper presents a hierarchy-based algorithm for incompressibility and hence unknot recognition. Section 9 explicitly limits its analysis to iteration counts in terms of maximal hierarchy length L and maximal pattern-complexity g . Proposition 9.1 bounds these iterations by $L(g + 1)^L$. That section does not bound both parameters in the input size and

states that the steps have not there been specified precisely enough for a running-time estimate [2, Section 9]. Thus the supplied talk’s announcement and the supplied paper’s detailed algorithm should not be silently identified with a fully specified implementation having the announced bound.

Here is the quantitative obstacle to transferring that bound to code. Even a hypothetical $g = O(n^2)$ only makes

$$L(g + 1)^L = \exp(O(\log L + L \log n)).$$

This is quasi-polynomial if an applicable effective length is $O(\log n)$; with $L = O(n)$ the same expression is $\exp(O(n \log n))$. Moreover, an iteration count must be multiplied by the bit cost of constructing and transforming the compressed states. Substituting a desired length bound without implementing and justifying the acceleration mechanism would not establish a complexity bound.

A genuine implementation of the advertised route would need compatible representations and executable routines for handle structures with boundary patterns, compressed normal surfaces and parallelity information, cutting and simplifying these structures, hierarchical multisurfaces, and the required Heegaard/Cheeger operations, with representation-size invariants connecting these routines. The supplied **fast/** code has none of these data types. Its frontier matchings are two-dimensional smoothing data, not compressed three-manifold hierarchies. Renaming them “multisurfaces” would not bridge the gap. Agol–Hass–Thurston’s compression techniques provide relevant machinery for such a different implementation [7], but are not a drop-in bound on this scanner.

Our implementation therefore retains the complete existing decision route and makes the improvements below. The restricted quasi-polynomial corollary in Section 6.3 follows from actual executable factorization, not from an assumed hierarchy routine. This assessment concerns the provided materials and this implementation; it is not a claim that the announced general result is false.

3 Execution architecture and exactness contract

The default decision path is:

$$\begin{aligned} & \text{validate} \longrightarrow \text{R1/R2 reduction} \longrightarrow \text{descending test} \\ & \longrightarrow \text{modular Alexander} \longrightarrow \text{capped Jones evaluations} \\ & \longrightarrow \text{exact Alexander} \longrightarrow \text{factored Khovanov ranks.} \end{aligned}$$

Each successful simplification or positive unknot test is exact. Each invariant filter may only return **KNOTTED**. An inconclusive invariant result proceeds to the next stage. The symbolic Alexander stage is retained both for compatibility and to handle modular collisions. It is not required for completeness once the Khovanov stage is present.

The Jones state cap defaults to 4096 and is a performance policy, not a topological assumption. Exceeding it skips that filter. In contrast, exhausting the overall time or homological-object budget prevents the requested complete calculation and returns **UNKNOWN**. The time budget is cooperative: it is polled at stage boundaries, scan construction steps, determinant pivots, factorization steps, and inside cancellation updates. A single expensive arithmetic operation can still overrun the requested wall time. An object cap is not a byte-level memory cap.

For the final criterion, Kronheimer–Mrowka prove that reduced Khovanov rank one detects the unknot [4]. The implementation uses $\mathbb{F}_2$: by the universal coefficient theorem, rational rank is no greater than $\mathbb{F}_2$ dimension. The rational reduced homology has nonzero Euler characteristic specialization and therefore positive dimension. Thus $\dim_{\mathbb{F}_2} \overline{\text{Kh}}(K) = 1$ forces rational rank one, and the detector applies. The converse follows from the explicit unknot complex. Section 6.2 explains why halving the unreduced rank is valid over $\mathbb{F}_2$, including degree by degree.

4 Exact one-sided modular rejection filters

4.1 Evaluating the Alexander minor without polynomial elimination

Let $F_D(t) \in \mathbb{Z}[t]$ be the determinant of the same $(n-1) \times (n-1)$ Fox minor used by the original implementation. Its entries have degree at most one, so

$$\deg F_D \leq n-1.$$

For a knot this minor represents the Alexander polynomial up to multiplication by a Laurent unit. In particular, if D represents the unknot, then

$$F_D(t) = \varepsilon t^k, \quad \varepsilon \in \{-1, 1\}, \quad 0 \leq k \leq n-1. \quad (1)$$

The polynomial matrix need never be constructed to test this necessary condition at a fixed value.

Proposition 4.1 (Sound modular rejection). *Fix a prime p and $a \in \mathbb{F}_p^\times$. If*

$$F_D(a) \notin \{\pm a^k : 0 \leq k \leq n-1\} \subseteq \mathbb{F}_p,$$

then D is not the unknot.

Proof. Evaluation modulo p is a ring homomorphism. If D were the unknot, (1) would put its evaluation in the displayed set. The contrapositive proves the claim. No assumption about avoiding collisions is needed for this implication. $\square$

The implementation uses $p = 1000000007$ and $a = -1, 2$, in that order. At -1 the allowed set is just $\{-1, 1\}$; many knots are rejected at this first evaluation. At 2 the whole allowed set is retained, rather than incorrectly demanding the value be ± 1 . A monomial t^k with $k > 0$ is still a unit in the Laurent ring.

The numeric Fox matrix is built directly over $\mathbb{F}_p$. Modular Gaussian elimination costs $O(n^3)$ field operations and $O(n^2)$ entries of storage. Only nonzero entries of the current pivot row need updates. The prime is fixed, so there is no coefficient swell. A zero determinant is also outside the unit set and is a valid rejection. When $n = 1$, the minor is empty and its determinant is one, as required. A zero-crossing diagram is handled before this test.

4.2 Jones evaluations as a matching-valued transfer computation

A PD row (a, b, c, d) has smoothings

$$0 : (a, b)(c, d), \quad 1 : (a, d)(b, c).$$

For $A \neq 0$, put $\delta = -A^2 - A^{-2}$. If $s \in \{0, 1\}^n$ and $c(s)$ is the number of circles in its complete smoothing, define the raw state sum

$$Z_D(A) = \sum_s A^{n-2|s|} \delta^{c(s)}. \quad (2)$$

This convention assigns δ to a crossing-free circle. With the compatible writhe convention, the normalized bracket is

$$J_D(A) = \delta^{-1} (-A^3)^{-w(D)} Z_D(A). \quad (3)$$

It equals one on the unknot. These are the usual bracket-state construction and writhe correction, with an explicit normalization chosen to match the input ports [6].

The important acceleration is aggregation before homology. At any partial scan, store one coefficient vector per boundary matching, rather than a chain complex with separate objects in all degrees. Attaching one smoothing closes c circles and changes the boundary matching. Multiply the coefficient by A or A^{-1} and by δ^c , and sum all contributions having the same resulting matching. The implementation evaluates at $A = 2$ and $A = 3$ simultaneously in $\mathbb{F}_p$.

Proposition 4.2 (Frontier recurrence). *After any prefix of the scan, the coefficient of a boundary matching is the sum of the partial-resolution weights that produce that matching, including the weights of all circles already closed. At the empty final boundary the coefficient is $Z_D(A)$.*

Proof. Initially there is one empty partial resolution of weight one. A new crossing partitions extensions into its two smoothings. Each extension multiplies the old weight by the appropriate $A^{\pm 1}$ and by one δ for each new closed circle. Future gluing depends only on the remaining matching, so summing coefficients for identical matchings loses no information. Induction gives the invariant. At the end every smoothing circle has closed and all crossing weights have been included, giving (2). $\square$

Both chosen evaluation points have nonzero δ . Therefore all inverses in (3) exist. If either normalized value differs from one, the knot is nontrivial. If both equal one, the result is inconclusive. In particular, the program does not assume that the Jones polynomial detects the unknot, nor that two evaluations determine the polynomial.

Let S be the largest retained state count. Apart from ordering, the transfer algorithm costs $\text{poly}(n, w)S$ for maximum boundary size w , with two evaluations a constant factor. For an arbitrary scan, a safe combinatorial bound on the number of matchings is $(w-1)!! = 2^{O(w \log w)}$, with the empty matching counted as one. A Catalan bound requires an appropriate single-disc cyclic-boundary condition; the greedy implementation does not establish that condition. It would be unsound to substitute it merely because the original link diagram is planar. With the default fixed state cap, the filter has only polynomial attempted work before it either completes or falls through.

4.3 Writhe and independently checkable rejection records

The original `signs()` routine gives local Fox-calculus signs relative to the under-port direction $a \rightarrow c$. These are not in general the oriented crossing signs. The new `writhe()` follows the actual traversal and uses the incoming under and over ports together. In its convention, pairs $(0, 3)$ and $(2, 1)$ have sign $+1$; the other two possibilities have sign -1 . A positive braid generator has writhe -1 in this particular input convention. Rotating a PD row by two ports must leave the writhe and normalized bracket unchanged; this is a regression test. Replacing the Fox signs themselves would instead have changed the existing Alexander matrix convention, so they are retained.

A modular rejection result contains the reduced PD, the R1/R2 reduction trace, and the exact field witness. `verify_rejection` first replays the legal moves on the original input, checks the resulting diagram, then recomputes the minor or bracket and checks the inequality. Tampering with the witness, prime, input diagram, or reduction trace is rejected. This is a reproducible certificate checker, not a formally verified kernel and not an independent implementation of the invariant algorithm. It intentionally does not claim to verify arbitrary Khovanov output transcripts.

5 Faster cobordism arithmetic and cancellation

5.1 The retained local algebra

The scanner is based on Bar-Natan’s local cancellation approach [3]. Boundary objects are perfect matchings. A morphism is an $\mathbb{F}_2$ combination of dot-subsets of the circles obtained by gluing the source and reversed target matchings. Delooping a closed circle replaces it with two summands. Arithmetic uses the Frobenius algebra

$$A_0 = \mathbb{F}_2[x]/(x^2), \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x, \quad \epsilon(1) = 0, \quad \epsilon(x) = 1.$$

The geometric composition routine in the baseline is retained as a reference path. Its repeated reconstruction of glued surfaces is avoidable when the same local data recur many times in the complex.

5.2 Every unit is its own inverse

For a matching m with r arcs, its endomorphism ring is

$$R_m \simeq \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2).$$

The baseline already recognizes units by a nonzero constant term. The new code uses the following stronger simplification.

Lemma 5.1. *If $f \in R_m$ has constant coefficient one, then $f^{-1} = f$.*

Proof. Write $f = 1 + u$, where u has zero constant coefficient. Every positive-degree square-free monomial squares to zero. In characteristic two, cross terms in the square of their sum cancel, so $u^2 = 0$. Thus $f^2 = (1 + u)^2 = 1$. $\square$

This argument applies to arbitrary combinations of monomials; it does not depend on retaining a quantum grading or on f being homogeneous. In the same ring, multiplication is particularly cheap: two dot-subsets with an intersection multiply to zero; otherwise they multiply to their union, with duplicates cancelled modulo two. The implementation uses this path when all three matchings in a composition agree. Zero and identity morphisms have immediate shortcuts.

5.3 Bounded memoization and immutable coefficients

Morphism values used as memoization keys and cached results are frozen. General composition is cached by both morphisms and all three boundary matchings. The entire local crossing transformation is cached by the input morphism, source and target matchings, smoothing choices, boundary, and crossing ports. This avoids recomputing the same geometric/delooping transformation for different copies of a homological object. A fresh outer dictionary is returned where the calling code expects mutable indexing, while cached coefficient values remain immutable.

The entry limits are 16384 for circle decompositions, 16384 for smoothing gluing, 65536 for composition, and 32768 for crossing transformations. These are entry bounds, not byte bounds: a large morphism is still large. Internal cached geometry is treated as read-only. `clear_caches()` and `cache_info()` support controlled experiments and release of retained state between runs.

5.4 Sparse, lazy Markowitz-style cancellation

Suppose an invertible differential entry $\phi : b \rightarrow c$ is selected. For each remaining incoming $\delta : a \rightarrow c$ and outgoing $\gamma : b \rightarrow f$, cancellation replaces the direct entry by

$$d'_{af} = d_{af} + \gamma\phi^{-1}\delta. \quad (4)$$

The minus sign of the general Schur complement is a plus over $\mathbb{F}_2$. The pair b, c is then deleted. Bar-Natan's cancellation lemma identifies the resulting complex as chain-homotopy equivalent to the original [3, Lemma 4.2].

The number of potential updates associated with a pivot is

$$\mu(b, c) = (|\text{in}(c)| - 1)(|\text{out}(b)| - 1).$$

The baseline uses a LIFO list of available units. The optimized default instead uses a heap with priority $(\mu, \text{degree sum}, b, c)$. On extraction it checks that both objects and the unit still exist, recomputes the current priority, and reinserts if stale. New invertible entries created by (4) are inserted.

This is deliberately described as a *lazy Markowitz-style heuristic*, not an exact global minimum-degree or minimum-fill algorithm. Some entries whose degrees have decreased can retain stale larger keys until popped. That affects performance, not validity: every actually cancelled pivot is an extant unit and therefore supports the same exact cancellation lemma. Between cancellations, only finitely many stale entries require refreshing. A cancellation removes two objects, so the process terminates. A persistent unit cannot be lost: it was present initially or inserted when created, and any stale live copy is refreshed rather than silently discarded.

Caching and pivot selection solve different problems. Caching makes a repeated update cheap; pivot selection can avoid millions of updates altogether. The large five-braid ablation in Section 9 separates these two effects. The original `compositions` statistic is kept for comparability: it increments twice per incoming/outgoing update pair. It is not a count of expensive cache misses or necessarily of actual geometric reconstructions.

6 Certified diagrammatic connected-sum factorization

6.1 Recognizing a valid cut without a prime-decomposition oracle

Traverse the diagram once and record the double-occurrence word of its crossing indices. Search proper cyclic intervals in which every crossing appearing in the interval appears twice. Such an interval has exactly two traversed cut edges. The implementation also computes the faces of the planar rotation system and requires the cut edges to have the same two *distinct* incident faces. It favors a balanced crossing partition among the cuts found.

Proposition 6.1 (Sound diagrammatic splitting). *A cut satisfying these tests yields a connected-sum decomposition of the knot. Identifying the two cut-edge labels separately in each side gives valid PDs for the factors, with the original crossings partitioned between them.*

Proof. The interval condition assigns every crossing wholly to one side and gives two edges joining the two parts. The face condition makes their dual edges a simple two-edge cycle: one arc can be drawn in each of the two distinct faces, joining their intersection points with the cut edges. Their union is a Jordan curve on the projection sphere, avoids all vertices, and meets the diagram transversely in exactly those two points. It separates the two crossing sets. Pushing this curve to a

sphere in three-space gives the usual connected-sum sphere meeting the knot twice. Closing each resulting arc across its disc corresponds exactly to identifying the two cut labels in its factor PD. The code additionally validates both resulting PDs. $\square$

The procedure is conservative. It does not find every topological connected-sum sphere after arbitrary diagram changes and is not a prime-decomposition algorithm. Failure to find a cut means only that this implementation will scan that piece directly. An explicit stack processes the decomposition tree without Python recursion-depth dependence. Each cut records the local PD, the two edge labels, the local crossing partitions, and their relation to the original crossing indices.

For a node with m crossings, each of $2m$ interval starts can be scanned in $O(m)$ set operations, giving $O(m^2)$ word-RAM work. There are fewer than $2n$ nodes, so $O(n^3)$ is a conservative total bound. Favoring balance helps in practice but is not used to assert that every decomposition tree is balanced.

6.2 Why the rank product and degree convolution are exact

We include the algebraic details because it would be incorrect to multiply unreduced ranks directly, or to halve arbitrary coefficient-field ranks. Let $C(D)$ be the cube complex over $\mathbb{F}_2$, with the basepoint away from crossings, and let X multiply by x on the circle through that basepoint. The reduced complex is $\tilde{C}(D) = \text{im } X = \ker X$.

Let ν act on each enhanced state by the sum of the operations replacing one x label by 1. A local check against multiplication and comultiplication gives

$$\nu d = d\nu, \quad Xd = dX, \quad \nu X + X\nu = 1.$$

For example, applying the total derivation to $\Delta(1) = 1 \otimes x + x \otimes 1$ gives two copies of $1 \otimes 1$, hence zero; on $\Delta(x)$ it gives $\Delta(1)$. These identities are the standard characteristic-two splitting mechanism [5, Section 3.2].

Lemma 6.2 (Unreduced/reduced splitting). *After forgetting quantum grading, $C(D)$ is the direct sum of two copies of $\tilde{C}(D)$ as a complex preserving cube degree. Consequently*

$$\dim \text{Kh}^h(D; \mathbb{F}_2) = 2 \dim \widetilde{\text{Kh}}^h(D; \mathbb{F}_2).$$

Proof. For $r \in \ker X$, the identity $X\nu r = r$ shows that $\nu : \tilde{C} \rightarrow C$ is a chain-map section of $X : C \rightarrow \tilde{C}$. Every $c \in C$ decomposes as $c = (c + \nu Xc) + \nu Xc$, with the first term in $\ker X$. The two summands intersect trivially since $X\nu$ is the identity on $\tilde{C}$. Both maps preserve cube degree. Taking homology proves the rank identity. Quantum shifts distinguish the two copies, but are not being reported. $\square$

Lemma 6.3 (Connected-sum product). *For diagrammatic factors D_1, D_2 with their basepoints at the joining arcs,*

$$\tilde{C}(D_1 \# D_2) \cong \tilde{C}(D_1) \otimes_{\mathbb{F}_2} \tilde{C}(D_2)$$

in raw cube degree, after forgetting quantum grading. Hence their reduced rank polynomials multiply.

Proof. A smoothing of the connected sum is a pair of smoothings with the two marked circles joined. In the reduced complex the joined circle is labelled x . This gives a basis bijection with pairs of enhanced states each having marked label x . At every crossing the saddle occurs in one factor only. The multiplication and comultiplication formulas above show that this bijection intertwines the differentials, including saddles involving the marked circle. The tensor differential has no sign distinction over $\mathbb{F}_2$. The Künneth formula over a field now identifies homology with the tensor product of factor homologies. Since the crossing sets are partitioned, cube degrees add. $\square$

Write $H_i(z) = \sum_h \dim \text{Kh}^h(D_i; \mathbb{F}_2) z^h$. The implementation computes

$$H_D(z) = 2 \prod_i \frac{H_i(z)}{2}. \quad (5)$$

The division is coefficientwise and checked for evenness. Multiplication is ordinary sparse integer convolution. Identical normalized factor PDs share a memoized rank result. The program never builds the tensor-product generators. It returns an exact integer total and a degree histogram with at most $n + 1$ entries.

6.3 Bounds and a genuine asymptotic improvement

Theorem 6.4 (Complexity of the delivered algorithm). *Let n_i be the crossing counts of the diagrammatic factors actually found, and $b = \max_i n_i$. The uncapped exact rank routine has a conservative bound*

$$T(D) \leq \text{poly}(n) + \sum_i 2^{O(n_i)} \leq \text{poly}(n) + n 2^{O(b)}.$$

The default complete recognizer has the same form after its polynomial preprocessing and fixed-cap filters. In particular, the general bound is $2^{O(n)}$.

Proof. Consider an unfactored piece of m crossings. Its explicit resolution cube has 2^m states and at most $m + 1$ smoothing circles per state, hence at most 2^{2m+1} enhanced states. Delooping and scanning cannot increase the number of objects beyond an exponential-size expanded state description; cancellation only deletes objects. There are exponentially many possible dotted basis terms at a boundary of $O(m)$ points, so every morphism can be stored and composed in $2^{O(m)}$ work. The number of cancellation updates is bounded by a polynomial in the exponential number of objects and entries. Lazy heap operations and bounded memoization do not change this exponential upper bound.

The decomposition is polynomial. Factor calculations are sequential and sum to $\sum_i 2^{O(n_i)}$. Intermediate histogram coefficients have $O(n)$ bits because all dimensions are bounded by an exponential-size cube. There are $O(n)$ degrees and $O(n)$ factors; even straightforward integer convolution therefore costs a polynomial number of bit operations. This proves the first bound, and at most n nonempty factors give the second. Modular elimination is polynomial; the fixed-cap Jones attempt is polynomial; the retained symbolic Alexander calculation has polynomial degree and coefficient-bit growth for its linear integer matrix entries. These stages do not increase the general exponential bound. $\square$

Corollary 6.5. *On diagrams for which the found maximum factor size is $O(\log n)$, the delivered algorithm runs in polynomial time. If that maximum is $O(\log^2 n)$, it runs in $n^{O(\log n)}$ time.*

This is a bound on *found diagrammatic factors*, not merely on the abstract prime factors of the knot. It does not provide a quasi-polynomial bound for a large prime knot or a diagram hiding its connected-sum structure.

The family

$$D_k = \sigma_1^3 \widehat{\sigma_2^3 \cdots \sigma_k^3} \quad \text{on } k + 1 \text{ strands}$$

is a concrete exponential-to-polynomial improvement for the *full rank* API. It is a visible connected sum of k trefoils, has $n = 3k$, and has reduced rank 3^k . The baseline ends with $2 \cdot 3^k$ separate homology objects and therefore takes at least $\Omega(3^k)$ object work. The new routine finds three-crossing factors and evaluates (5) compactly in polynomial time. This does not claim an exponential speedup

of default *recognition* on that family: the baseline can already reject these knots by Alexander polynomial.

The same family shows why scan boundary size alone is not a valid complexity parameter for this implementation. It admits the measured width-four scan, yet has exponentially growing homological multiplicity. Boundary matching types and the number of copies of each type are different quantities. The original width-only explanation is therefore explicitly not repeated as a guarantee.

7 Faster polynomial preprocessing

7.1 An equivalent greedy order in $O(n \log n)$ per start

The baseline greedy rule repeatedly scans every unprocessed crossing, preferring more shared boundary edges, then more self-loops, then the lower crossing index. For one start this takes $O(n^2)$ comparisons. The new implementation builds the crossing adjacency lists once. Each crossing has at most four incident edges. When a crossing is processed, only its still-unprocessed neighbors have their shared-edge scores increased. A lazy heap stores

($-\text{shared edges}$, $-\text{self-loops}$, index).

There are $O(n)$ score changes and heap entries, so a start costs $O(n \log n)$ time and $O(n)$ space. The original start sequence and tie-breaking are retained exactly, including the baseline's behavior when the requested number of starts does not divide n . Randomized differential tests compare *every* start against the unchanged baseline. Thus improvements in the raw scan comparisons are not explained by silently choosing easier crossing orders.

7.2 All descending basepoints by one interval sweep

Fix an oriented traversal of its $2n$ crossing encounters. A diagram is descending from a basepoint if the overpass is the first encounter with every crossing. For a crossing with over-encounter o and under-encounter u , the bad starting positions form the cyclic interval $(o, u]$: those starts see u first. Add one to each such interval using a cyclic difference array. A prefix sum gives the number of bad crossings at every start. Exactly the zero positions are valid descending basepoints.

Repeat for the reversed traversal and choose the smallest valid dart, matching the previous API. This takes $O(n)$ work and space instead of retraversing the entire diagram from every candidate dart. The fact that a descending knot diagram represents the unknot is unchanged. The R1/R2 reduction strategy itself is not redesigned; deadline checks are added between its decreasing moves.

8 Validation and its limits

The delivered regression suite has 30 test methods. The original 19 methods are retained; only expectations of default rejection-method names and explicit filter-disabling options are adapted where the pipeline has changed. The exact unchanged original tests remain in `baseline/tests/` and also passed. The new tests cover algebraic identities, randomized oracle comparisons, scan-order equivalence, descending-basepoint equivalence, factor products and degree histograms, port rotations, label permutations, invalid orders, resource budgets, cache bounds, and tampered certificates. The original difficult Alexander-trivial named examples are retained.

A separate exhaustive script enumerates every word of length 2, 4, or 6 over $\{\sigma_1^{\pm 1}, \sigma_2^{\pm 1}\}$ whose closed three-braid has one component. These are diagram words, not distinct knot isotopy classes.

For each word it compares the direct scanner and factored scanner with an independent dense reduced cube, checks $d^2 = 0$ during the direct scan, compares the Jones transfer calculation to a full independent smoothing-state sum, and checks the default recognizer’s verdict. For every modular rejection it recomputes the certificate.

Word length	One-component words checked
2	8
4	160
6	2688
Total	2856

There are 1712 unknot words and 1144 rejected nontrivial words in this set, with zero discrepancies. The dense cube reference is adapted from the original archive’s independent test oracle. It enumerates enhanced states, explicitly builds saddle differentials, and performs $\mathbb{F}_2$ column elimination without using the scanner’s cobordism composition or gluing. The Jones oracle separately uses an edge-disjoint-set computation for each complete smoothing, without the frontier `glue` function.

The large 36-crossing example is beyond the dense reference run performed here. Its two completed cancellation strategies agree on the *entire* raw homological degree histogram. Six checked R2 reductions give a 24-crossing diagram whose optimized calculation, with d^2 checks, again has reduced rank 2949. Its independently computed exact Alexander determinant is also 2949. These are useful consistency checks; the determinant agreement is not an independent full-rank proof and is not reported as one. No Lean formalization or third-party full-rank computation is claimed.

9 Same-machine experiments

9.1 Methodology

The measurements used CPython 3.13.5, Linux x86-64, and an AMD EPYC 9V74 processor. Each sample runs in a fresh subprocess, pinned to the same eligible logical CPU when the operating system supports affinity, with `PYTHONHASHSEED=0`. Samples are serial. Module import, input JSON parsing, and initial conversion into a diagram are outside the timed region. The public API’s own validation, ordering, and any enabled factorization are inside it. Caches are cleared before timing. Engine order is alternated between repetitions. Except for the long five-braid case, tables show medians of three samples. Millisecond-scale ratios should not be interpreted as high-precision measurements of hardware-independent constants.

The baseline is the actual supplied source, not a reconstructed approximation. The **rank** task uses no R1/R2 simplification. Factoring is disabled in generic raw-rank comparisons and enabled only in the connected-sum suite. Completed rank runs must agree degree by degree; completed recognition runs must agree in verdict. Timeouts are censored observations, not fabricated finishing times. The harness records both the cooperative budget and any parent-process watchdog expiration. Source fingerprints are recorded at measurement time; the final manifest separately identifies the delivered files.

9.2 Raw homology calculations

Instance	n	Reduced rank	Baseline (ms)	New (ms)	Speedup
Conway	11	33	34.402	11.951	$2.88\times$
Kinoshita–Terasaka	11	33	38.206	13.160	$2.90\times$
Hard unknot	8	1	2.321	2.754	$0.84\times$
$T(3, 5)$	10	7	9.731	9.599	$1.01\times$
Unknot braid	40	1	8.290	3.494	$2.37\times$
$T(2, 41)$	41	41	82.195	25.089	$3.28\times$
Random four-braid	21	25	259.337	81.672	$3.18\times$
Random four-braid	31	45	301.728	70.107	$4.30\times$
Random four-braid	41	109	954.437	249.370	$3.83\times$
Random six-braid	31	85	515.203	88.843	$5.80\times$

Most nontrivial examples improve by roughly three to six times, while the tiny hard-unknot raw-rank case regresses. This is consistent with saving repeated algebra and updates where there is enough work to amortize heap, validation, and caching overhead. No universal constant-factor speedup is claimed.

9.3 The large five-braid: separating caching from cancellation

The exact word, also saved as `examples/random_5_braid_36.json`, is

```
[-4,-3,4,4,-3,2,1,1,-1,-1,3,-4,-1,3,2,2,-2,-1,
-3,2,3,-3,-4,-4,-3,3,1,3,-2,-4,-4,3,-3,3,-4,-2]
```

All rows below scan this same original 36-crossing diagram, with the same crossing order, without factoring or Reidemeister simplification.

Engine	Seconds	Reduced rank	Update counter
Supplied baseline	> 60	not obtained	not completed
New arithmetic/caches, LIFO	49.576	2949	71 475 304
New arithmetic/caches, lazy fill-aware	1.110	2949	190 930

The baseline raised its time-budget exception after 60.073 seconds; it did not complete in the budget. The resulting conservative comparison is more than $60/1.110 > 54$ times faster within this test budget. The completed LIFO ablation versus the new pivot order gives about 44.7 times improvement, with all other optimized machinery held fixed. Each of these expensive configurations was measured once.

Both completed scans have the same maximum pre-cancellation object count 17694, maximum post-cancellation count 5898, 16884 eliminations, and boundary width eight. What changes dramatically is the number of attempted cross updates, by a factor of about 374.4. The entire rank histogram agrees. Process memory high-water marks, including interpreter/import overhead, are approximately 276.2 MiB for cached LIFO and 127.1 MiB for the new pivot heuristic. The baseline’s censored peak is not its eventual peak and is not used as a completed-run memory comparison.

9.4 Default recognition, including regressions

Instance	n	Baseline (ms)	New (ms)	Speedup
Conway	11	34.036	1.610	21.14×
Kinoshita–Terasaka	11	38.233	1.563	24.47×
Hard unknot	8	1.915	2.735	0.70×
$T(3, 5)$	10	0.661	0.266	2.48×
Trefoil	3	0.097	0.119	0.82×
Unknot braid	40	6.889	6.995	0.98×
Determinant-one grid	10	0.808	0.281	2.87×
Scrambled unknot grid	6	0.302	0.283	1.07×

Conway and Kinoshita–Terasaka have trivial Alexander polynomials in the supplied examples. The new exact Jones evaluations reject them without constructing a homological differential, explaining the roughly 21–24-fold gains. The hard unknot reaches the complete fallback and pays for filters that do not reject it; it is about 43 percent slower on this small input. The tiny trefoil also shows a small overhead. The table includes these regressions rather than only the wins.

9.5 Compact products instead of exponentially many objects

Factors	Crossings	Baseline (ms)	New (ms)	Peak objects	Reduced rank
2	6	2.562	1.911	18	3^2
4	12	15.440	2.611	18	3^4
6	18	176.910	3.008	18	3^6
8	24	1305.710	3.638	18	3^8
20	60	not run	6.618	18	3^{20}
50	150	not run	22.922	18	3^{50}
100	300	not run	65.597	18	3^{100}

For eight trefoil factors, the old rank routine peaks at 39366 pre-cancellation objects and ends with 13122 unreduced homology objects. The new routine needs at most 18 pre-cancellation objects in a factor. At 100 factors it still uses that same object peak and returns

$$3^{100} = 515377520732011331036461129765621272702107522001.$$

The input has 300 crossings. A full degree histogram, not just the scalar product, is computed. Baseline runs above eight factors were not performed in this experiment and are marked “not run,” not timed out or assigned an extrapolated time.

9.6 Ordering alone

n	Baseline (ms)	New (ms)	Speedup
128	3.995	0.199	20.1×
256	15.995	0.365	43.8×
512	65.135	0.725	89.9×
1024	251.053	1.629	154.1×

These inputs are closures of $\sigma_1 \cdots \sigma_n$ on $n + 1$ strands. The timed task is a single greedy order, not knot recognition. The output orders agree exactly. The observed scaling is consistent with the proved replacement of quadratic rescanning by incremental heap updates, but the complexity claim rests on the operation count, not on fitting these four timings.

10 Using and reproducing the deliverable

No installation is required. From the archive root:

```
cd fast
python -m fastunknot recognize examples/conway.json --output conway-proof.json
python -m fastunknot verify examples/conway.json conway-proof.json
python -m fastunknot khovanov examples/random_5_braid_36.json --no-factors
python -m fastunknot recognize examples/hard_unknot_8.json
```

Python 3.10 or later is required. For a normal installed console command, `python -m pip install ./fast` can be run from the archive root. There are no runtime package dependencies. The same source commands work in PowerShell.

For a controlled fallback comparison, disable both invariant families:

```
python -m fastunknot recognize examples/conway.json --no-alexander --no-jones
```

The `-no-alexander` option disables modular and symbolic Alexander, but not Jones. The `khovanov` subcommand bypasses all recognition preprocessing; `-pivot lifo` selects the cached old pivot strategy. An explicitly supplied library scan order must be a full permutation and disables factorization.

From the archive root, the validation and measurement commands are:

```
python -m unittest discover -s fast/tests -v
python tools/validate_exhaustive.py
python tools/benchmark.py
```

The last command takes several minutes because it deliberately repeats the baseline time-limited case and cached-LIFO ablation. `-only conway` or `-only ordering_` limits the requested cases. The exhaustive script writes the case counts, rank histogram, certificate count, result-stream digest, and large-input consistency checks to `results/exhaustive.json`.

For library use:

```
from fastunknot import Diagram, recognize, khovanov_rank

d = Diagram.from_braid(3, [1, -2, 1, -2])
answer = recognize(d, seconds=30, max_objects=200_000)
print(answer.status, answer.method)

homology = khovanov_rank(d.pd, factor_connected=True)
assert homology["reduced_rank"] == 5
```

The public JSON contracts retain the distinction between UNKNOT, KNOTTED, and UNKNOWN. A completed knot verdict has exit code zero; a resource limit has code three; invalid input or options have code two. Certificate verification uses zero for a verified modular rejection and one for failure.

The report can be rebuilt from `report/` with two runs of `pdflatex -interaction=nonstopmode -halt-on-error unknot_speedup.tex`. No bibliography download, shell escape, external figure, or network connection is needed. `changes.patch` exposes the implementation changes relative to the original source. The original 600-second Windows benchmark is retained only under `baseline/results/`; all new comparisons are under `results/`.

11 Remaining limits and concrete next implementation targets

The generic worst case is still exponential. Diagrammatic factors can all be large; the Jones filter can be inconclusive or exceed its cap; Alexander can be trivial for a nontrivial knot; the homological complex can have exponentially many surviving objects; and bounded caches do not bound total RAM. The lazy pivot heuristic is not optimal and can lose on particular inputs. The test suite is extensive but finite, and the certificate checker shares its invariant implementation with the producer.

Further practical work can target a compact integer encoding of matchings and dot monomials, moving cache-key construction and finite-field elimination into compiled kernels, and improved separator-based order heuristics. Those changes would need new ablations because changing crossing order and cancellation order simultaneously can obscure the source of a gain. A complete general quasi-polynomial hierarchy implementation is a distinct target: it requires the three-manifold representations and compressed-operation proofs identified in Section 2, not just a faster scalar matrix reducer.

The present deliverable is therefore a tested, usable speed-up with exact one-sided filters, a much faster difficult raw-rank example, and a genuine exponential-to-polynomial improvement for compactly decomposable rank problems. Its bounds and evidence are deliberately stated at the level actually supported by the source code and measurements.

References

- [1] Marc Lackenby, *Unknot recognition in quasi-polynomial time*, talk slides, 2021. Supplied as docs/quasipolynomial-talk.pdf; 109 physical pages. The announcement is on physical pages 13–14; speed-up ingredients on page 72; final flowchart on page 109. <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [2] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. Supplied TeX source and figures in docs/arXiv-2607.23350v1/; the complete PDF is available from arXiv. In particular, Section 9 and Proposition 9.1, physical pages 34–35. <https://arxiv.org/abs/2607.23350>.
- [3] Dror Bar-Natan, *Fast Khovanov Homology Computations*, Journal of Knot Theory and Its Ramifications **16** (2007), no. 3, 243–255; arXiv:math/0606318. <https://arxiv.org/abs/math/0606318>.
- [4] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346 (2010). <https://arxiv.org/abs/1005.4346>.
- [5] Alexander N. Shumakovitch, *Torsion of the Khovanov homology*, Fundamenta Mathematicae **225** (2014), 343–364; arXiv:math/0405474v2 (2018). In particular, the X, ν identities and Corollary 3.2.C. <https://arxiv.org/abs/math/0405474>.
- [6] Louis H. Kauffman, *Combinatorial Knot Theory and the Jones Polynomial*, arXiv:2204.12104 (2022), the originator’s exposition of the bracket state model. <https://arxiv.org/abs/2204.12104>.
- [7] Ian Agol, Joel Hass, and William P. Thurston, *The Computational Complexity of Knot Genus and Spanning Area*, arXiv:math/0205057 (2002). <https://arxiv.org/abs/math/0205057>.